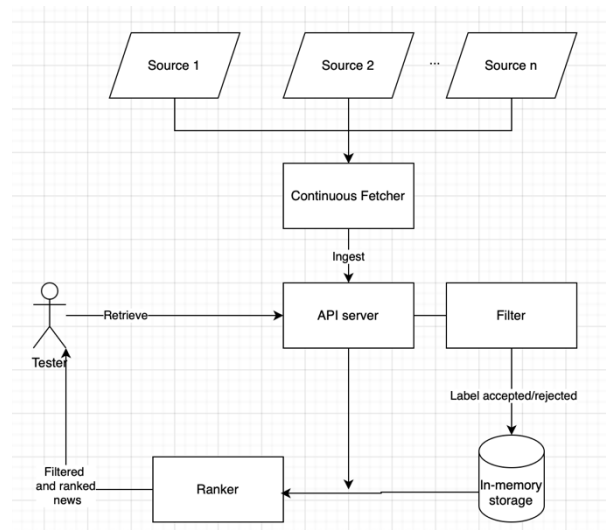


NEWSFEED ASSIGNMENT - REFLECTION

ARCHITECTURE DESIGN

The solution is structured around three core components:

1. **Ingestion Layer:**
 - a. Events are ingested through a FastAPI application that exposes a single /ingest endpoint. Incoming payloads are validated using Pydantic models, ensuring strict type checking and schema robustness.
 - b. A lightweight continuous fetcher loop periodically collects events from all external sources (Reddit + RSS feeds) and forwards them to the same /ingest endpoint. This keeps ingestion logic centralized so that both the automated test harness and the fetcher rely on the exact same pipeline.
 - c. For simplicity, the fetcher runs on a basic Python loop.
2. **Filtering & Scoring:** Filtering is implemented using a rules-based relevance scoring system designed for clarity, interpretability, and deterministic behavior:
 - a. The system defines three sets of signal categories (security terms, outage terms and bug terms) and then each incoming event is scored based on matching keywords, and some heuristics to boost for instance appearance of keywords in the title.
 - b. Scoring weights are manually fixed intentionally so they can be easily adjusted to refine the system.
 - c. This approach ensures stable, transparent filtering, and makes false positives/negatives easy to reason about. The keyword lists are intentionally simple and can grow organically over time as operational understanding improves.
3. **Storage & Retention:** Events are stored in-memory using a small internal storage abstraction:
 - a. This choice keeps the project lightweight and avoids unnecessary infrastructure for the assignment.
 - b. A retention policy removes events older than a configurable window. This prevents unbounded memory growth and ensures that /retrieve only returns relevant, recent items.



ASSUMPTIONS

Several practical assumptions were made:

1. The system does not need to store long-term historical data. Events older than 7 days are discarded. Storing a full historical archive for model training or analytics was considered out of scope.
2. The ingestion does not require strict real-time guarantees. A polling interval of ~5 minutes is acceptable to meet the “continuous” requirement.
3. The API runs in a controlled, non-public environment and it is not exposed publicly as it would be vulnerable to common web attack (there is no auth, rate limiting, or security controls beyond basic Pydantic validation)
4. The expected data volume is limited, so in-memory storage will not encounter memory pressure within the retention window

FUTURE WORK

1. **Filtering**
 - Introduce ML-based relevance detection (TF-IDF, embeddings, logistic regression, XGBoost).
 - Add semantic similarity checks (e.g., using Sentence Transformers).
 - Weight events based on source authority or historical trust.
2. **Storage**
 - Implement dynamic retention so that critical events are stored longer and irrelevant elements removed earlier.
 - Move from in-memory storage to for instance a serverless option like DynamoDB
3. **Ingestion**
 - Replace the “manual” loop by a proper scheduler (e.g. a cloud-native cron service like AWS EventBridge)
 - Add a per-source rate limiting (explain what does this mean)

Bonus Question: How would you evaluate the efficiency and correctness of your news retrieval and filtering process?

Efficiency:

- I would add timing logs in the /ingest and /retrieve endpoints to evaluate the time to process a batch and to return the results
- Use a small load-testing tool like Locust to send multiple /ingest calls with synthetic batches and hit /retrieve concurrently.

Correctness:

- Build a small curated set of labelled events (ordered by importance) that simulate realistic inputs and assess whether the system outputs the desired order.
- Use “negative control” events unrelated to IT incidents (for instance adding as a source the feed of a sports newspaper, cooking magazine,...) to confirm they are correctly filtered out.

FINAL NOTE FOR THE REVIEWER

I did this assignment while travelling in Indonesia where Reddit is blocked. Therefore, even If I added the code to fetch from an IT subreddit, I couldn't test that part, that's the reason why is not added as a source to the news fetcher. Hope it is not a problem! 😊